E110 • PROJECT 08

Hack32

```
001011  00011  00001  00010  00000000001
010001  00100  00001  0000000000010111
001100  00011  00010  00001  00000000010
100000  00000000000000000000000001011010
```

A 32-bit reimagining of the Hack architecture

32 registers • 3 instruction formats • load/store • hardware multiply

PRESENTED BY

Ethan Silwal
Kyle Lin
Curtez Gentry
Triston Abrams

GROUP 13

Spring 2026

How we thought about the redesign

We didn't want to throw Hack away — we wanted to keep what works and fix what hurts. Three questions guided every choice we made.

Q1 What was painful about base Hack?

- Only 2 registers (A and D) — every non-trivial program juggles values through memory
- 16-bit instructions leave no room for register fields or rich opcodes
- No multiply instruction — even simple math compiles to slow software loops
- Memory-mapped A register tangles arithmetic with addressing

Q2 What did we want to keep?

- The simplicity — fixed-width instructions, clean fetch-decode-execute
- The teaching value — every chip should still be buildable from NAND and DFF
- Compatibility of mindset — anyone who learned Hack should recognize Hack32 immediately

Q3 What design did we steal from?

- RISC-V's three-format encoding (R / I / J) — proven, simple, compiler-friendly
- MIPS's load/store separation — arithmetic on registers only, memory only via LW/SW
- RISC-V's R0-as-zero trick — gives us free constants and unconditional branches

Four changes that define Hack32

01

32 registers

R0 hardwired to zero. R1–R31 general purpose. Five-bit fields select any register, any time.

02

3 formats

R-type, I-type, J-type instruction encodings — each 32 bits — covering ALU, memory, and control flow.

03

Load / store

Memory only moves through LW and SW. ALU operates strictly on registers, like RISC-V.

04

Hardware MUL

A native 32-bit multiplier replaces the software loops that base Hack needed for multiplication.

Each piece is described on the slides that follow.

32 registers, each 32 bits wide

R0	R1	R2	R3	R4	R5	R6	R7
R8	R9	R10	R11	R12	R13	R14	R15
R16	R17	R18	R19	R20	R21	R22	R23
R24	R25	R26	R27	R28	R29	R30	R31

R0 hardwired to 0

R1–R28 general purpose

R29 SP • R30 RA • R31 scratch

DESIGN NOTES

Five-bit register fields

$2^5 = 32$ — every field in every format selects any of the 32 registers cleanly.

R0 = 0

Reads always return zero; writes are discarded. Useful for compares, NOPs, and unconditional branches.

Calling convention

R29 acts as the stack pointer, R30 holds return addresses, R31 is a free scratch slot for codegen.

Any reg, any address

Load and store accept any of the 32 registers as a base address — no dedicated A register.

Three 32-bit instruction formats

R-TYPE

register-register arithmetic & logic

`ADD, SUB, AND, OR, MUL`



I-TYPE

immediate, load, store, branch

`ADDI, LW, SW, BEQ, LUI`



J-TYPE

unconditional jump

`JMP, JAL`



All instructions are exactly 32 bits — fixed width keeps the fetch and decode logic simple.

Arithmetic and logical operations

INSTR	TYPE	OPERATION	EXAMPLE	NOTES
ADD	R	rd = rs1 + rs2	ADD R3, R1, R2	two's-complement add
SUB	R	rd = rs1 − rs2	SUB R3, R1, R2	uses inverted-B path
AND	R	rd = rs1 & rs2	AND R3, R1, R2	bitwise
OR	R	rd = rs1 | rs2	OR R3, R1, R2	bitwise
XOR	R	rd = rs1 ^ rs2	XOR R3, R1, R2	bitwise
MUL	R	rd = rs1 × rs2	MUL R3, R1, R2	**NEW — hardware multiply**
ADDI	I	rd = rs1 + imm_{16}	ADDI R3, R1, #5	sign-extended immediate
LUI	I	rd = imm_{16} ≪ 16	LUI R3, #0x1234	build the upper 16 bits

Logical and arithmetic operations are pure register-to-register; only LW and SW touch memory.

Worked examples — what each op actually does

Each example shows the same two 8-bit operands feeding the ALU, with the result the operation produces. Bit patterns are shown in green where bits are set.

ADD — rd = a + b

a:	0	0	0	0	1	1	0	0
b:	0	0	0	0	1	0	1	0
→	0	0	0	1	0	1	1	0

12 + 10 = 22

SUB — rd = a − b

a:	0	0	0	0	1	1	0	0
b:	0	0	0	0	1	0	1	0
→	0	0	0	0	0	0	1	0

12 − 10 = 2

AND — rd = a & b

a:	0	0	0	0	1	1	0	0
b:	0	0	0	0	1	0	1	0
→	0	0	0	0	1	0	0	0

bitwise — only bits set in both

OR — rd = a | b

a:	0	0	0	0	1	1	0	0
b:	0	0	0	0	1	0	1	0
→	0	0	0	0	1	1	1	0

bitwise — bits set in either

XOR — rd = a ^ b

a:	0	0	0	0	1	1	0	0
b:	0	0	0	0	1	0	1	0
→	0	0	0	0	0	1	1	0

bitwise — bits set in exactly one

MUL — rd = a × b

a:	0	0	0	0	1	1	0	0
b:	0	0	0	0	1	0	1	0
→	0	1	1	1	1	0	0	0

12 × 10 = 120 (NEW)

Five of these are standard ALU work — the sixth, MUL, is the new feature we built.

Load / store: the only path to memory

Hack32 is a strict load/store machine. Arithmetic instructions never touch memory — they only operate on registers. Memory is read or written only through LW and SW.

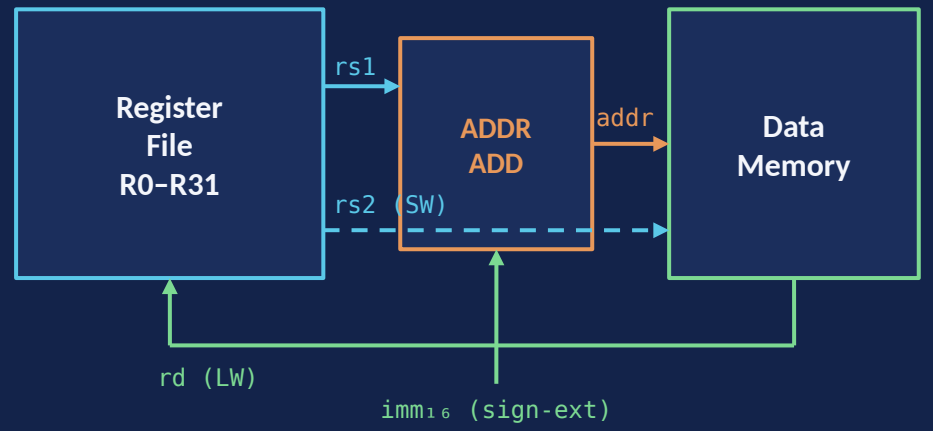
LW / SW ENCODING

```
LW   rd, offset(rs1)
       rd ← MEM[rs1 + sign_extend(offset)]

SW   rs2, offset(rs1)
       MEM[rs1 + sign_extend(offset)] ← rs2
```

Any of R0–R31 can serve as the base register, and the 16-bit immediate offset is sign-extended for negative displacements.

DATA PATH



solid = read • dashed = write • green = memory side

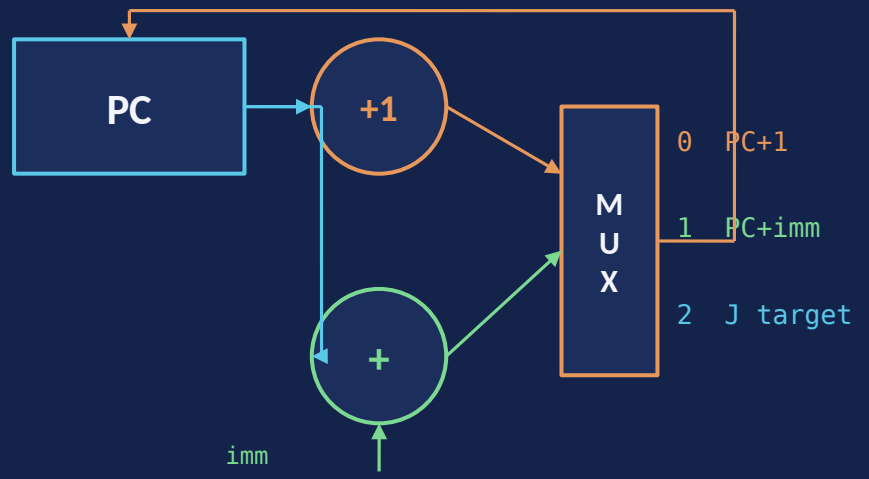
Conditional branches and jumps

CONDITIONAL & UNCONDITIONAL

INSTR	FMT	BEHAVIOR
BEQ	I	if rs1 == rs2 → PC = PC + imm
BNE	I	if rs1 != rs2 → PC = PC + imm
BLT	I	if rs1 < rs2 → PC = PC + imm
BGE	I	if rs1 >= rs2 → PC = PC + imm
JMP	J	PC = address (26-bit target)
JAL	J	R30 = PC + 1; PC = address

BEQ R0, R0, target acts as an unconditional relative branch — handy because R0 is always zero.

PC UPDATE PATH



Compare unit drives the mux select for branches; J-type instructions force select = 2.

Loading constants into any register

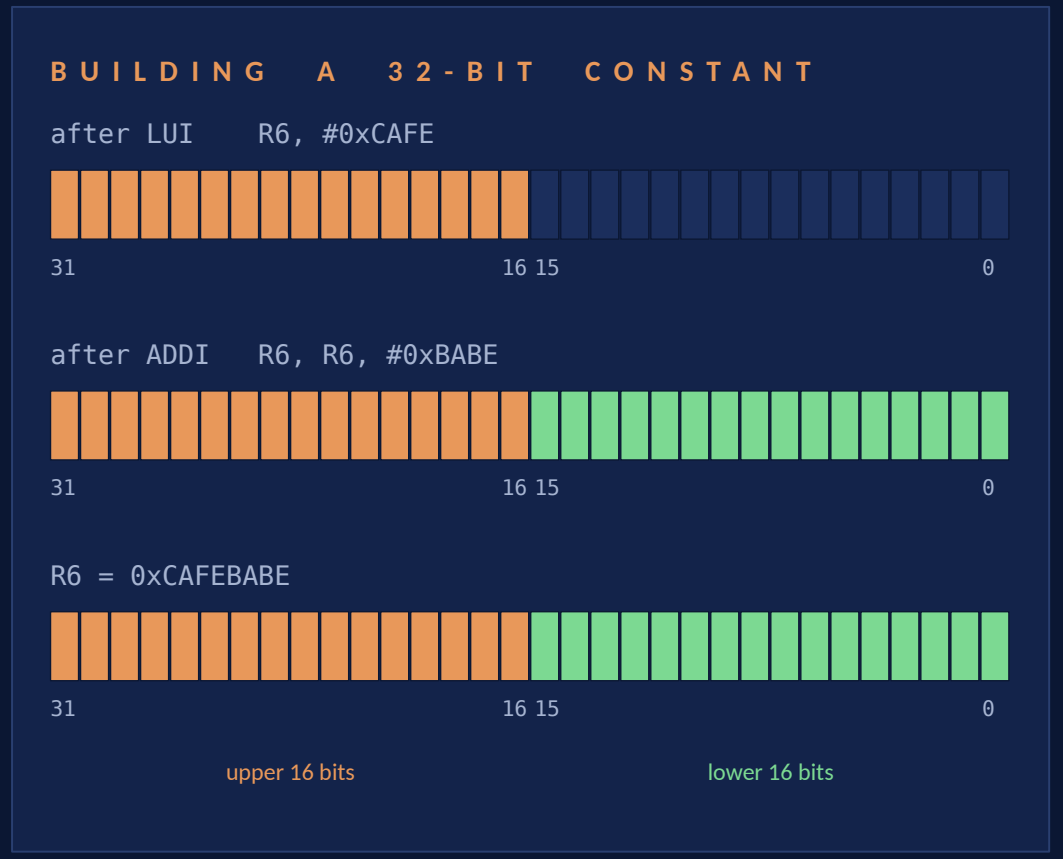
ADDI handles the common case — adding a small signed constant to a register, so any of R1–R31 can be initialized in a single instruction by adding to R0.

For full 32-bit constants the assembler emits LUI + ADDI: LUI fills bits 31:16, then ADDI fills bits 15:0. Two instructions, any 32-bit value, any register.

EXAMPLES

```
ADDI R5, R0, #42
  ; R5 ← 42

LUI  R6, #0xCAFE
ADDI R6, R6, #0xBABE
  ; R6 ← 0xCAFEBABE
```



Hardware multiply (MUL)

Why MUL?

Base Hack has no multiply instruction — programs compile multiplication into a software loop of repeated additions, which is slow and bloats code size.

Multiplication shows up everywhere in real programs: array indexing, graphics transforms, and basic arithmetic libraries. Putting it in hardware makes Hack32 measurably faster on common workloads.

~32×

worst-case speed-up over a 32-bit software multiply loop, since hardware MUL completes in a single dispatch instead of up to 32 iterations.

IMPLEMENTATION

```
MUL  rd, rs1, rs2
```

Format

R-type, with a unique funct code so existing decoder logic extends cleanly.

Algorithm

Shift-and-add multiplier — for each bit of rs2, conditionally add a shifted rs1 into a partial product.

Result width

Lower 32 bits of the 64-bit product land in rd. Overflow detection is left for a future revision.

ALU change

One new functional unit feeds the ALU output mux; control logic selects it on the MUL funct code.

INSPIRED BY: *RISC-V Unprivileged ISA Manual, Ch. 12 — "M" Extension for Integer Multiplication and Division.* Our MUL matches the spec: *"places the lower XLEN bits in the destination register."*

A small Hack32 program

Sum of squares $1^2 + 2^2 + \ldots + N^2$

This short routine touches every category in the rubric: load immediate, register arithmetic, MUL, conditional branch, and unconditional jump. N is loaded from memory; the running sum is written back at the end.

WHAT EACH LINE EXERCISES

- load immediate
- register arithmetic
- hardware multiply (NEW)
- branch / jump
- load / store

hack32_sum_of_squares.s

```
      LW   R1, 0(R0)        ; R1 = N
      ADDI R2, R0, #1       ; i  = 1
      ADDI R3, R0, #0       ; sum = 0
loop:
      MUL  R4, R2, R2       ; R4 = i*i
      ADD  R3, R3, R4       ; sum += i*i
      ADDI R2, R2, #1       ; i++
      BLT  R2, R1, loop     ; if i < N
      BEQ  R2, R1, loop     ; include N
      SW   R3, 4(R0)        ; store sum
      JMP  done
done: ADDI R0, R0, #0       ; halt
```

How Hack32 meets every requirement

32

32-bit instructions

Fixed-width 32-bit encoding across R, I, and J formats.

+−

Arithmetic / logical

ADD, SUB, AND, OR, XOR, ADDI, plus the new MUL.

↔

Memory load / store

LW and SW with sign-extended offsets and any base register.

?

Conditional branches

BEQ, BNE, BLT, BGE update PC based on register comparisons.

→

Unconditional jumps

JMP and JAL with 26-bit targets; JAL saves the return address.

#

Load immediates

ADDI loads small constants; LUI + ADDI builds full 32-bit values.

32R

32 registers

R0 hardwired zero; R1–R31 general purpose with 5-bit fields.

★

Any reg as address

Load and store accept any of the 32 registers as the base.

★

New feature

Hardware MUL — native shift-and-add multiplier in the ALU.

Thank you — questions?

Group 13 • Ethan Silwal • Kyle Lin • Curtez Gentry • Triston Abrams

REFERENCES: *RISC-V Unprivileged ISA Manual (Ch. 12, M Extension); Nisan & Schocken, The Elements of Computing Systems (base Hack architecture)*